

Library Management API

Technical Review – Questions and Answers

Candidate Technical Clarifications

Project: Library Management API
Repository: github.com/NPrisci/library-managment-api
Date: September 2, 2026
Framework: Laravel / PHP

This document provides detailed answers to the technical questions raised during the review of the project.

Introduction

Thank you for the detailed technical questions regarding the Library Management API.

I reviewed the points raised in the questions and went back through the implementation to verify the behavior of the application. During this review, I also corrected several issues that had been identified, including the integration of the generic controllers, pagination collection, authorization policy, price storage, and request validation.

The corrected implementation has been pushed to the project repository:

<https://github.com/NPrisci/library-management-api>

The answers below describe both the original behavior of the application and the corrections that have now been applied.

Question 1 – BorrowingController::extend()

Question:

Walk me through the `extend()` method in `BorrowingController`. What does it currently do when someone calls `PATCH /borrowings/{id}/extend`?

The `extend()` method is leftover code from an abandoned feature. While running the tests, I got errors because the `due_date` field does not exist in the borrowings table specified by the exercise. I therefore removed the `due_date` field and the related database code, but I forgot to remove the `extend()` method.

When someone calls `PATCH /api/v1/borrowings/{id}/extend`, Laravel resolves the borrowing through route model binding. The method checks that the authenticated user owns the borrowing and validates the provided data. However, since `due_date` is no longer part of the database design, the method does not actually update or save anything. It eventually loads the book and user relationships and returns the existing borrowing with a 200 response.

So, this endpoint is not part of the required functionality of the exercise and should be removed.

Question 2 – CategoryController

Question:

`CategoryController` has five empty methods but is registered with `Route::apiResource`. What happens when someone calls `GET /api/v1/categories`?

When someone calls `GET /api/v1/categories`, the `apiResource` route sends the request to the `index()` method of `CategoryController`.

However, my `index()` method was initially empty and didn't return anything. Therefore, no categories were returned. The route existed, but the endpoint was not implemented.

I have now corrected this by implementing the `GenericController` as required in step 32 and making `CategoryController` delegate to it. Now the endpoint properly returns the categories.

Question 3 – GenericController, CustomPaginationCollection and BorrowingPolicy

Question:

You wrote GenericController, CustomPaginationCollection and BorrowingPolicy. Can you point me to where each one is actually used in the running application?

Yes. I have just corrected these parts.

GenericController is now used by AuthorController and CategoryController through inheritance (class AuthorController extends GenericController and class CategoryController extends GenericController) to handle their common CRUD operations.

CustomPaginationCollection is now used by the endpoint where I applied the custom pagination format, with return new CustomPaginationCollection(\$authors);. It returns fields such as itemsReceived, curPage, nextPage, itemsTotal, and pageTotal.

BorrowingPolicy is now used by BorrowingController for authorization. I added `$this->authorize('return', $borrowing)` before calling the borrowing service, so only the user who borrowed the book can return it.

These were corrections I made after reviewing the requirements, so they are now actually connected to the running application.

Question 4 – Inline Authorization vs BorrowingPolicy

Question:

In BorrowingController you check `$borrowing->user_id !== $user->id` inline, but you also wrote a BorrowingPolicy with a `return()` method. Why two mechanisms for the same rule?

Initially, I checked the user's ownership directly inside BorrowingController with `$borrowing->user_id !== $user->id`.

Later, I implemented BorrowingPolicy as required by the exercise and changed `returnBook()` to use `$this->authorize('returnBook', $borrowing)`.

So the two mechanisms exist because the inline check in the controller is from my initial implementation, while the Policy is the cleaner and more centralized approach I added afterward. Ideally, I would refactor the remaining inline check to use the Policy as well.

Question 5 – Book Scopes and BookController

Question:

What happens if you call `Book::byAuthor(5)` in Tinker? And why does `BookController::index()` use raw `where()` clauses rather than the scopes?

After correcting the scope name, `Book::byAuthor(5)` works correctly. It uses the local scope and filters books where `author_id` is 5. In my database, it returns an empty collection because there are no books for author 5.

`index()` uses raw `where()` clauses because it handles multiple optional filters dynamically. The scopes could be used instead, and that would make the code more reusable and readable.

Question 6 – Price Storage and PriceCast

Question:

Your books migration has `$table->decimal('price', 10, 2)` with `// $table->integer('price');` commented out above it. Talk me through that decision, and given PriceCast multiplies by 100 on write, what value actually ends up in that column for a \$12.99 book?

I initially used `$table->decimal('price', 10, 2)` when I was testing the application, and I had `$table->integer('price')` commented out.

After testing, I corrected my PriceCast and BookTest so that the price is stored as an integer in the database, in cents, as required by the exercise.

I then corrected the migration as well, so I now use:

```
$table->integer('price')->default(0);
```

With this setup, when I send a price of \$12.99, the PriceCast multiplies it by 100 before saving it, so the value stored in the database is 1299 cents.

Question 7 – PriceCast and BookResource

Question:

Explain how PriceCast and BookResource::price_formatted work together.

PriceCast handles the conversion between the price used in the application and the value stored in the database, while BookResource formats the price for the API.

With the `set()` method, PriceCast converts the application price into an integer representing cents before storing it in the database. For example, \$12.99 becomes 1299.

With the `get()` method, PriceCast does the opposite: it takes 1299 from the database and converts it back to 12.99 for the application.

Then, BookResource uses this value and returns it in the JSON response as `price` and `price_formatted`. For example:

```
{ "price": 12.99, "price_formatted": "$12.99" }
```

PriceCast handles the data conversion, and BookResource::price_formatted handles the presentation of that price in the API.

Question 8 – BookBorrowed Event and Database Transaction

Question:

In `BorrowingService::borrowBook()`, why is `BookBorrowed::dispatch()` inside the `DB::transaction()` closure rather than after it? What would go wrong if it were outside?

`BookBorrowed::dispatch()` is inside the transaction because the event is related to the borrowing and the book update. This keeps the event within the same transaction as the database changes. If the event were dispatched outside the transaction, it could be triggered even if the transaction failed or was rolled back, which could create an inconsistency: the event would say that the book was borrowed, while the borrowing might not actually exist in the database.

Question 9 – BaseFormRequest

Question:

You wrote BaseFormRequest with a custom failedValidation(). Which of your request classes extend it, and what error format do StoreBookRequest and UpdateBookRequest return when validation fails?

Initially, they extended FormRequest directly, so they were not using my custom BaseFormRequest. I corrected both classes to extend BaseFormRequest. Now, when validation fails, they return a 422 JSON response with a message field and an errors object containing the validation errors.

Question 10 – LogRequestDuration and terminate()

Question:

The brief asked for a LogRequestDuration middleware with a terminate() method. It isn't in the repo, but I'd still like to know: what is terminate() for, and why does timing a request need it rather than doing the work in handle()?

handle() runs during the request lifecycle, before the response is sent. We can use it to record the start time and then call \$next(\$request) to continue processing the request. terminate() runs after the response has been sent to the client. It is useful for tasks such as logging the request duration without making the user wait for that logging work. So, for a LogRequestDuration middleware, I would record the start time in handle() and calculate and log the duration in terminate(). The important point is that terminate() allows the logging work to happen after the response has been sent.

Corrections Applied

Following the review, I went through the repository and applied the corrections discussed in the answers above.

The main corrections include:

- Corrected the integration of `GenericController`.
- Connected `AuthorController` and `CategoryController` to the generic CRUD implementation.
- Integrated `CustomPaginationCollection` into the relevant API endpoint.
- Integrated `BorrowingPolicy` into the borrowing authorization flow.
- Corrected the price storage implementation to use integer cents.
- Updated `StoreBookRequest` and `UpdateBookRequest` to extend `BaseFormRequest`.
- Corrected the custom validation error response.
- Reviewed the obsolete borrowing extension functionality.

The updated implementation has been pushed to GitHub.

[View the updated repository on GitHub](https://github.com/NPrisci/library-managment-api)

<https://github.com/NPrisci/library-managment-api>

Thank you for the detailed review and for the opportunity to clarify these implementation decisions.